

Name: Atul Kumar Reg. No: 12102801  
 Roll No: RD2110B61 Assignment: 01 Set: 3  
 Course code: CAP-615 Instructor: Mr. Kumar Vishal

Q1. Analyze the difference between for loop and advance for loop. How advance for loop is used in 2D array? write one example.

| Ans: Normal for Loop                                                                                            | Advance for Loop                                                                                    |
|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| ① This normal for loop is present from JDK1.                                                                    | ① This advance for loop is present from JDK5.                                                       |
| ② In for loop, we can increase the counter as per our requirement by using $i = i + x$ ( $x = 1, 2, 3, \dots$ ) | ② In advance for loop we can increase the counter in a sequentially order (always increased by one) |
| ③ Using for loop, we can iterate in both decrement or increment.                                                | ③ In this loop, we can iterate only in increment order.                                             |
| ④ In this loop we can iterate on any container object.                                                          | ④ We can only iterate on that container by using this loop to implement the iterable interface.     |

⑤ In this loop, we can replace elements at any specific index.

⑥ In this for loop, we can print array elements either in the original order or in reverse order.

⑦ Syntax:-

```
for(int i=0; i<10; i++)  
{  
    // Statement  
}
```

⑧ Ex:-

```
int[] x = {1, 2, 3};  
for(int i=0; i<x.length; i++)  
{  
    System.out.println(x[i]);  
}
```

Output

1  
2  
3

In this loop, we don't have access to the index, so we can't replace elements at any specific index.

In this loop, we can print array element only in the original order, not in reverse order.

Syntax:-

```
for(data type variable  
    : array / collection)  
{  
    // Statement  
}
```

Ex:-

```
int[] x = {1, 2, 3};  
for(int a:x)  
{  
    System.out.println(a);  
}
```

Output

1  
2  
3



How advance loop is used in 2D array?

Ans: For using advance for loop in 2D array we need two advance loops (nested advance loop) for iteration.

Ex: // Program to find average

```
public class Average
```

```
{
```

```
    public static double getAvg(int[][] a)
```

```
    {
```

```
        double total = 0;
```

```
        for (int[] innerArray : a)
```

```
        {
```

```
            for (int val : innerArray)
```

```
            {
```

```
                total = total + val;
```

```
            }
```

```
        }
```

```
        return total / (a.length * a[0].length);
```

```
    }
```

```
    public static void main (String k[])
```

```
    {
```

```
        int[][] theArray = {{80, 90, 70}, {80, 80, 75}};
```

```
        System.out.println(getAvg(theArray));
```

```
    }
```

```
}
```

Output

69.16666666666667

Q2. With the help of example define Encapsulation Features in Java.

Ans: Encapsulation Features in Java

Encapsulation in Java is a process of wrapping code and data together into a single unit. We can create a fully encapsulated class in Java by making all the data members of the class private. So we can say that encapsulation is also a kind of "data hiding". Thus encapsulation acts as a protective shield around the data and prevent the data from unauthorized access by the outside world.

Ex: // Program to find area of a rectangle  
class Area {

int length;

int breadth;

Area(int length, int breadth) { // constructor

this.length = length;

this.breadth = breadth;

}

```

public void getArea() { // method
    int area = length * breadth;
    System.out.println("Area: " + area);
}
}

class Main
    public static void main (String k[])
    {
        Area rectangle = new Area(5, 6);
        rectangle.getArea();
    }
}

```

## Output

Area: 30

Here, the fields and methods can be accessed from other classes as well. This is encapsulation. We can just keep similar codes together.



Q3. Define and differentiate the following pairs:

a) the logical  $\&\&$  and  $\|\|$  operators.

Ans: The logical AND operator ( $\&\&$ ) returns true if both operands are true and return false otherwise. The operands are implicitly converted to type bool before evaluation and the result is of type bool.

The Logical OR operator ( $\|\|$ ) for a set of operands is true if and only if one or more of its operands is true. It is typically used with boolean (logical) values.

Logical  $\&\&$  (AND)

$x = 6$

$y = 3$

$x < 10 \&\& y > 1$

Return True

Logical  $\|\|$  (OR)

$x = 6$

$y = 3$

$x == 5 \|\| y == 5$

Return False

b) a break and a continue statement

Ans: Break statement: Break statement or break keyword is used to terminate and transfer the control to the next statement when encountered, inside a loop or switch case statement.

- i) A break can appear in both switch and loop (for, while, do) statements.
- ii) Break causes the switch or loop statement to terminate the moment it is executed.
- iii) When a break statement encountered, it terminates the block and gets the control out of the switch or loop.
- iv) A break statement inside the nested loop allows the termination of the innermost loop and the control remain inside the outermost loop.

Continue Statement: The continue statement is used for the termination of the current iteration and not the whole loop. Continue is not used to terminate the execution of loop. Continue can only appear in loop (for, while, do) statements.



- i) continuing statement inside the nested loop allows the skip of the current iteration and execution of the next iteration of the innermost loop.
- ii) when a continue statement is encountered, it gets the control to the next iteration of the loop.

### break statement

```
while (expression)
{
    statement(s);
    if (condition for break)
    {
        break;
    }
}
break comes here
statement(s);
```



### continue statement

```
while (expression)
{
    // body
    if (condition for continue)
    {
        continue;
    }
    // body
}
```

